

CIRA

Static Security Audit

Discovery & Inventory Report

Security Auditor • Deepapraakash Rajaram

Audit status DISCOVERY & INVENTORY PHASE	Assessment type Static source-code analysis only
Repository CIRA-main monorepo	Evidence basis User-supplied CIRA Deep Static Security Audit Report

Scope limitation. No active exploitation, database mutation, remediation, or runtime testing was performed. This report records static source-code observations and hypotheses for later controlled validation; it does not establish runtime exploitability unless expressly stated as confirmed by code.

Prepared 25 August 2026

1. Executive Summary

A deep static analysis of the CIRA application repository (CIRA-main) was conducted. The application consists of a Node.js/Express backend, a FastAPI Python backend, a React frontend, and an Electron desktop client, backed by PostgreSQL and MongoDB.

The supplied source identifies code-confirmed issues including an unauthenticated student dashboard, hardcoded/default administrative credentials, and a non-expiring password-reset OTP. It also identifies strongly supported authorization, upload/resource-management, and business-logic hypotheses, plus potential deployment- and platform-dependent issues requiring runtime validation.

Interpretation rule: “Confirmed by code” means the cited implementation is present in the static source reviewed. It does not substitute for runtime evidence of reachability, exposure, or exploitability. “Strongly supported” and “Potential” findings remain validation candidates.

2. Scope and Assessment Boundaries

In scope	CIRA monorepo: Backend-Core, Backend-AI, Frontend-Web, Desktop Client, PostgreSQL, MongoDB, Docker configuration, and Prisma seed artifacts.
Performed	Static review and discovery/inventory documentation based solely on the supplied audit source.
Not performed	Active exploitation, database mutation, remediation, destructive testing, credential brute forcing, network attacks, or runtime testing.
Intended next phase	Controlled runtime validation in an owned local development environment or explicitly authorized isolated staging environment using non-production test accounts and data.

3. Attack-Surface Inventory

Backend-Core	Node.js, Express, Prisma	REST API; DOCX/XLSX uploads; JWT authentication; database queries
Backend-AI	Python, FastAPI	Internal REST API; ML inference; NLP grading
Frontend-Web	React, Vite	DOM XSS; client-side routing; LocalStorage
Desktop-Client	Electron	IPC handlers; kiosk-mode escapes; Node integration
Databases	PostgreSQL, MongoDB	Exposed ports; default credentials; raw queries

4. Data-Flow and Authorization Analysis

Faculty → Student Results. Faculty can evaluate students through *faculty.controller.ts* → *evaluateStudent*. The supplied audit reports role checking without verification that the target `student_id` belongs to a department/section mapped to that faculty member.

Student → Quiz Responses. *student-exam.controller.ts* → *saveResponse* accepts `questionId` and upserts a `QuizResponse` without verification that the question belongs to the quiz linked to the active attemptId.

File uploads. `req.file` (DOCX) flows through Multer with no configured size limit, then disk storage, then *mammoth.convertToHtml* and memory. The source identifies a risk of resource exhaustion from a deeply nested/compressed DOCX.

Password resets. A 6-digit `verificationCode` is generated and stored without a timestamp. The source notes the code does not expire and `reset-password` rate limiting was not observed.

5. Finding Catalogue

The following entries preserve the supplied finding identifiers, severity labels, statuses, affected components/files, code evidence, impacts, and validation objectives. Safe validation approaches are added as operational guardrails and do not change the original classification.

CIRA-001 (AUTH-01): Unauthenticated Student Dashboard

Severity	CRITICAL
Status	CONFIRMED BY CODE
Confidence classification	Confirmed by code
Affected component	Backend-Core
Affected files / functions	<code>student-dashboard.routes.ts</code> ; <code>student-dashboard.controller.ts</code>
Security property	Authentication & Authorization
Evidence description	The <code>authenticate</code> and <code>authorize</code> middlewares are commented out on the <code>/dashboard</code> route. The controller hardcodes <code>rollNumber: ch.sc.u4cse24141</code> to fetch data.
Potential impact	Total exposure of a specific student's PII and analytics to any unauthenticated internet user.
Source validation objective	Send an unauthenticated GET request to <code>/api/v1/student-dashboard/dashboard</code> and verify it returns a 200 OK with the hardcoded student's data.
Safe Phase 3 validation approach	In the local or authorized isolated staging environment, use only a synthetic student record and observe whether an unauthenticated request is rejected or returns data. Capture the HTTP status and a redacted response; do not use real student records.

CIRA-005 (BL-01): Hardcoded/Default Admin Credentials

Severity	HIGH
Status	CONFIRMED BY CODE
Confidence classification	Confirmed by code
Affected component	Backend-Core
Affected files / functions	<code>prisma/seed.ts</code>
Security property	Credential Management
Evidence description	The seed script hardcodes an administrator email and a known default password during database seeding. The credential values are not reproduced in this report.
Potential impact	If the seed script is run in staging/production, attackers can trivially log in as an administrator.

Source validation objective	Attempt to log into the application using the administrator email and default password identified in prisma/seed.ts.
Safe Phase 3 validation approach	Inspect seed behavior and use a disposable locally seeded environment only. Confirm whether a known default credential is created, then immediately reset or discard the test environment. Never attempt these credentials on production or any third-party system.

CIRA-014 (AUTH-04 [NEW]): Non-Expiring Password Reset OTP

Severity	HIGH
Status	CONFIRMED BY CODE
Confidence classification	Confirmed by code
Affected component	Backend-Core
Affected files / functions	auth.controller.ts — forgotPassword, resetPassword
Security property	Cryptographic Lifecycle / Brute Force Prevention
Evidence description	forgotPassword generates a 6-digit numerical code and saves it to the user record. resetPassword checks whether the code matches but does not check an expiration timestamp. The source also notes lack of rate limiting.
Potential impact	Complete account takeover. An attacker can request a reset for any user and take indefinite time to guess the 6-digit code.
Source validation objective	Request a password reset for a test account, wait an extended period (for example, 24 hours), and verify the code is still accepted by the reset-password endpoint.
Safe Phase 3 validation approach	Use one owned test account. Request one reset, record timestamps, and later check whether the same code remains valid. Do not guess codes, test other users, or run high-volume requests; record only expiry behavior and response status.

CIRA-003 (BOLA-01): Faculty Resource-Level Authorization Bypass (IDOR)

Severity	HIGH
Status	STRONGLY SUPPORTED
Confidence classification	Strongly supported / requires runtime validation
Affected component	Backend-Core
Affected files / functions	faculty.controller.ts (evaluateStudent); quiz.controller.ts (evaluateAttempt)
Security property	Authorization (BOLA/IDOR)
Evidence description	The code trusts student_id, assessment_id, and attemptId supplied by the client to mutate database records (prisma.result.update, prisma.quizAttempt.update). No cross-reference query ensures the authenticated faculty member is mapped to the target student's department/section or owns the quiz.
Potential impact	A malicious or compromised faculty account can modify grades and feedback of any student in the university.
Source validation objective	Authenticate as Faculty A (Department X). Attempt to evaluate an attempt ID belonging to Student B (Department Y). Verify if the database mutation succeeds.
Safe Phase 3 validation approach	Seed two synthetic departments, two faculty accounts, and two students in an isolated database. Make one bounded cross-boundary request and compare the target record before and after. Stop after observing the authorization result; restore or discard the test database.

CIRA-006 (UP-01): DOCX Parser Unbounded Resource Consumption

Severity	MEDIUM / HIGH
Status	STRONGLY SUPPORTED
Confidence classification	Strongly supported / requires runtime validation
Affected component	Backend-Core
Affected files / functions	quiz.routes.ts; quiz.controller.ts (uploadDocxParser)
Security property	Input Validation / Resource Management
Evidence description	const upload = multer({ dest: 'uploads/' }); handles upload and the file is passed directly to mammoth.convertToHtml({ path: filePath }). No multer limits: { fileSize: ... } is configured globally or on this route; Mammoth parses DOCX ZIP/XML content in memory.
Potential impact	Application denial of service through memory exhaustion from a ZIP bomb or massive upload.
Source validation objective	Upload a specially crafted large/highly compressed DOCX file and monitor Node.js process memory usage and response time.
Safe Phase 3 validation approach	Use a disposable local service with conservative host/container resource limits and a harmless oversized test document—not a compression bomb. Confirm whether the application rejects it cleanly and record peak memory, elapsed time, and service health.

CIRA-016 (BL-03 [NEW]): Cross-Quiz Response Injection

Severity	MEDIUM
Status	STRONGLY SUPPORTED
Confidence classification	Strongly supported / requires runtime validation
Affected component	Backend-Core
Affected files / functions	student-exam.controller.ts — saveResponse
Security property	Business Logic / Authorization
Evidence description	saveResponse accepts attemptId and questionId from the client and creates/updates a QuizResponse. It does not query whether questionId belongs to the quizId associated with attemptId.
Potential impact	A student could submit answers referencing question IDs from different or future exams, potentially corrupting database state or discovering answers via automated evaluation.
Source validation objective	Start Exam A. Submit a save-response request using the attemptId for Exam A but a questionId belonging to Exam B. Verify if the database successfully creates an orphaned response record.

Safe Phase 3 validation approach

Create two entirely synthetic quizzes and a test student in an isolated database. Submit one mismatched reference, inspect the resulting response record, then reset the test data. Do not use active assessments or real student accounts.

CIRA-002 (AI-01): Unauthenticated Backend-AI Service

Severity	HIGH (If reachable)
Status	POTENTIAL (Depends on network architecture)
Confidence classification	Potential / environment dependent
Affected component	Backend-AI
Affected files / functions	main.py
Security property	Not specified in supplied source
Evidence description	FastAPI binds to 0.0.0.0:8000 with no authentication middleware. CORS allows * by default if ALLOWED_ORIGINS is not set.
Potential impact	Potential direct access to Backend-AI functionality if the service is reachable.
Source validation objective	Attempt to access the analytics endpoint directly from an external network or a compromised student frontend.
Safe Phase 3 validation approach	From an explicitly authorized isolated network segment, verify intended reachability and authentication with a benign health/analytics request using synthetic data. Do not scan the public internet or test an environment without written authorization.

CIRA-004 (DB-01): Unauthenticated MongoDB Exposure

Severity	HIGH (If reachable)
Status	POTENTIAL (Depends on firewall/host routing)
Confidence classification	Potential / environment dependent
Affected component	Docker Configuration
Affected files / functions	docker-compose.yml
Security property	Not specified in supplied source
Evidence description	Ports 27017:27017, 27018:27018, and 27019:27019 are bound to the host. No MONGO_INITDB_ROOT_USERNAME is defined.
Potential impact	Potential unauthenticated database exposure if the host ports are reachable.
Source validation objective	Run an external port scan against the host infrastructure to see if 27017 is reachable from the internet.
Safe Phase 3 validation approach	Do not internet-scan. Review the authorized deployment configuration and, from a designated internal test host, perform a narrowly scoped connection check to the owned staging host. Confirm whether authentication is required without reading, writing, or enumerating data.

CIRA-009 (EL-01): Electron Kiosk Bypass

Severity	MEDIUM
Status	POTENTIAL
Confidence classification	Potential / target-OS dependent
Affected component	Desktop Client
Affected files / functions	desktop-client/main.js
Security property	Not specified in supplied source
Evidence description	The app attempts to block shortcuts (Cmd+Q, Alt+F4), but OS-level interactions—Task Manager, virtual desktops, and Windows key—might not be fully suppressed by standard Electron APIs.
Potential impact	Potential escape from kiosk mode on supported operating systems.
Source validation objective	Launch the desktop client on target Windows/macOS platforms and attempt to escape kiosk mode using OS-specific keystrokes.
Safe Phase 3 validation approach	Use an owned test workstation with no real exam session. Define the approved supported operating systems and verify a short, documented set of normal OS/window-management paths. Record whether containment meets the stated threat model; do not bypass institutional controls on production exam devices.

CIRA-019 (API-02 [NEW]): Missing Rate Limiting on External APIs

Severity	LOW/MEDIUM
Status	POTENTIAL
Confidence classification	Potential
Affected component	Backend-Core & Backend-AI
Affected files / functions	Routing files (no express-rate-limit or similar middleware observed)
Security property	Not specified in supplied source
Evidence description	No express-rate-limit or similar middleware was observed in routing files.
Potential impact	Potential service degradation or crash from rapid concurrent requests to expensive endpoints.
Source validation objective	Send rapid, concurrent requests to expensive endpoints (such as login, upload-docx, or NLP grading) to verify whether the server degrades or crashes.
Safe Phase 3 validation approach	Use a local or isolated staging environment with explicit load-test approval, strict request and time ceilings, monitoring, and a stop condition. Start with minimal concurrency and measure throttling and resource behavior; do not attempt to crash a service.

6. Severity and Status Summary

CIRA-001	CRITICAL	CONFIRMED BY CODE	Backend-Core
CIRA-005	HIGH	CONFIRMED BY CODE	Backend-Core
CIRA-014	HIGH	CONFIRMED BY CODE	Backend-Core
CIRA-003	HIGH	STRONGLY SUPPORTED	Backend-Core
CIRA-006	MEDIUM / HIGH	STRONGLY SUPPORTED	Backend-Core
CIRA-016	MEDIUM	STRONGLY SUPPORTED	Backend-Core
CIRA-002	HIGH (If reachable)	POTENTIAL (Depends on network architecture)	Backend-AI
CIRA-004	HIGH (If reachable)	POTENTIAL (Depends on firewall/host routing)	Docker Configuration
CIRA-009	MEDIUM	POTENTIAL	Desktop Client
CIRA-019	LOW/MEDIUM	POTENTIAL	Backend-Core & Backend-AI

7. Phase 3 — Controlled Runtime Validation Plan

Authorized locations only. Test only in (1) the user's own local development environment or (2) an explicitly authorized, isolated staging environment. Use non-production test accounts and synthetic data. Do not test public, third-party, university production, or other systems without clear authorization.

Preparation. Document approval and scope; isolate services and databases; seed synthetic identities, courses, quizzes, and records; ensure backups/snapshots or disposable data; enable application, database, and host monitoring; define request ceilings and stop conditions.

Evidence standard. For each test, record the finding ID, environment, test account, timestamp, redacted request/response metadata, expected versus observed authorization outcome, and pre/post state of only the relevant synthetic record. Capture logs and resource measurements where relevant. Do not retain real PII or reusable credentials in the evidence package.

Method. Confirm the narrow security property once or with the minimum needed repetitions. Prefer benign requests and configuration inspection. Do not brute-force credentials or OTPs, deliberately crash services, make uncontrolled concurrent requests, or scan beyond the authorized test boundary.

Decision rule. Mark a finding runtime-validated only when evidence demonstrates the behavior in the approved environment. If a control prevents the behavior, record the result as not reproduced in that environment; do not downgrade static evidence without reviewing the deployment and code path.

8. Where to Test and How

Local development	Preferred for initial confirmation. Run only services built from the CIRA repository, use a disposable local database, and seed synthetic accounts/data.
Isolated staging	Use only if explicitly authorized. Segment it from production, remove real student data, limit access to named testers, and confirm logging/rollback before starting.
Never in scope without authorization	Public endpoints, third-party hosts, production databases, institutional networks, other users' accounts, or live examination devices.
Testing approach	Use the per-finding "Safe Phase 3 validation approach" in Section 5. Validate one hypothesis at a time, collect redacted evidence, and restore/discard the test environment afterward.

Warning: This document is a defensive audit artifact. It is not permission to test systems outside the approved environments above. If environment ownership or authorization is uncertain, stop and obtain written approval before proceeding.

9. Closing Statement

This report is a static security audit / discovery and inventory record. Its findings are limited to what the supplied CIRA Deep Static Security Audit Report supports. Runtime validation, remediation, and regression testing remain separate, controlled phases.